

## Bases de Datos II

### Trabajo Práctico 7

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

[https://github.com/Lucenear/Tecnicatura\\_Programacion\\_UTN/tree/main/3er\\_Semestre/BBD\\_D\\_II](https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/BBD_D_II)

## Parte 1: Implementación con Driver Nativo

```
lucenear@lucenear:~/Proyectos/Facultad GIT/Tecnatura_Programacion_UTN/3er_Semestre/BBDD_II/TP_VII_Drivers_Conexion_ODM$ node driver_lab.js
Conectado exitosamente con Driver Nativo
Se insertaron 3 documentos.
IDs: {
  '0': new ObjectId('6a41c09b8a77d58eed2b092f'),
  '1': new ObjectId('6a41c09b8a77d58eed2b0930'),
  '2': new ObjectId('6a41c09b8a77d58eed2b0931')
}
lucenear@lucenear:~/Proyectos/Facultad GIT/Tecnatura_Programacion_UTN/3er_Semestre/BBDD_II/TP_VII_Drivers_Conexion_ODM$
```

## Parte 2: Implementación con Mongoose (ODM)

```
lucenear@lucenear:~/Proyectos/Facultad GIT/Tecnatura_Programacion_UTN/3er_Semestre/BBDD_II/TP_VII_Drivers_Conexion_ODM$ node mongoose_lab.js
Conectado exitosamente con Mongoose
Documento guardado: {
  nombre: 'Microcontrolador ESP32',
  stock: 50,
  _id: new ObjectId('6a41c0bd1628d8721a4b26'),
  createdAt: 2026-06-29T00:47:57.675Z,
  updatedAt: 2026-06-29T00:47:57.675Z,
  __v: 0
}
lucenear@lucenear:~/Proyectos/Facultad GIT/Tecnatura_Programacion_UTN/3er_Semestre/BBDD_II/TP_VII_Drivers_Conexion_ODM$
```

## Parte 3: Reflexión Final

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?

El driver de MongoDB actúa como un puente o "traductor" fundamental que permite la comunicación entre el código de la aplicación (en este caso, Node.js) y el motor de la base de datos MongoDB. Su función principal es tomar las instrucciones escritas en el lenguaje de programación (como los métodos insertMany, find, etc.) y traducirlas al protocolo binario de red nativo que MongoDB entiende, llamado MongoDB Wire Protocol. Luego, toma la respuesta de la base de datos y la convierte nuevamente en estructuras de datos manejables y utilizables por la aplicación.

2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?

El driver convierte los objetos nativos de JavaScript (y JSON) al formato BSON (Binary JSON). BSON es una serialización binaria que extiende el formato JSON tradicional añadiendo soporte para tipos de datos más complejos (como Date, ObjectId, Buffer o enteros de 64 bits). Este formato es mucho más eficiente para la transmisión por red y permite un procesamiento (como operaciones de búsqueda y lectura) extremadamente rápido por parte del motor interno de la base de datos.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?

Se optaría por usar el driver nativo en situaciones donde el rendimiento, la eficiencia y el control a bajo nivel son críticos, y donde no necesitamos forzar un esquema o validación estricta desde la aplicación. Algunas situaciones específicas incluyen:

- Procesamiento masivo de datos (Big Data): Inserciones masivas (bulk inserts) o migraciones de bases de datos. Un ODM como Mongoose añade una "sobrecarga" (overhead) al crear instancias y validar documento por documento. El driver nativo es muchísimo más rápido para estas operaciones.

- Microservicios de altísimo rendimiento y baja latencia: Donde cada milisegundo de respuesta cuenta y se requiere la comunicación más directa posible sin capas intermedias.
- Proyectos de datos muy dinámicos/irregulares: Si una aplicación saca el máximo provecho de la naturaleza schemaless (sin esquema) de MongoDB con datos cuya estructura muta constantemente, forzar un esquema restrictivo en Mongoose podría ser contraproducente.

### Codigo Parte 1: driver\_lab.js

```
const { MongoClient } = require('mongodb');

//URI
const uri = "mongodb://localhost:27017/";
const client = new MongoClient(uri);

async function run() {
  try {
    //Conexion
    await client.connect();
    console.log("Conectado exitosamente con Driver Nativo");

    const database = client.db('laboratorio');
    const collection = database.collection('sensores_robotica');

    //Array
    const sensores = [
      { tipo: "Sensor de Humedad", modelo: "DHT11", precision: "±5%" },
      { tipo: "Servo Motor", modelo: "MG996R", torque: "10kg/cm" },
      { tipo: "LED RGB", modelo: "WS2812B", voltaje: "5V" }
    ];

    //Insert
    const result = await collection.insertMany(sensores);
    console.log(`Se insertaron ${result.insertedCount} documentos.`);
    console.log("IDs:", result.insertedIds);

  } catch (error) {
    console.error("Error en driver nativo:", error);
  } finally {
    await client.close();
  }
}

run().catch(console.dir);
```

## Codigo Parte 2: mongoose\_lab.js

```
const mongoose = require('mongoose');

//Conexion
mongoose.connect("mongodb://localhost:27017/laboratorio")
  .then(() => console.log("Conectado exitosamente con Mongoose"))
  .catch(err => console.error("Error de conexión:", err));

//Esquema
const componenteSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true
  },
  stock: {
    type: Number,
    default: 0
  }
}, {
  timestamps: true // Genera createdAt y updatedAt
});

//Creacion modelo
const Componente = mongoose.model('Componente', componenteSchema);

//Inserta datos de prueba
async function crearComponente() {
  try {
    const nuevoComponente = new Componente({
      nombre: "Microcontrolador ESP32",
      stock: 50
    });

    const guardado = await nuevoComponente.save();
    console.log("Documento guardado:", guardado);

    // Cerrar conexión al finalizar
    await mongoose.disconnect();
  } catch (error) {
    console.error("Error al guardar:", error);
  }
}

crearComponente();
```